Relatório de Governança e Ética de IA

TaskAnalyzer — Fase 3 — Bootcamp III

Repositório: github.com/PedroVargas1204/sdd-task-analyzer
Documento gerado automaticamente por `tools/gerar_pdfs.py` a partir de `docs/relatorio_governanca_etica.md`
Data de geração: 25/10/2026

Identificação

Campo	Informação
Projeto	TaskAnalyzer — módulo de métricas de produtividade
Repositório	github.com/PedroVargas1204/sdd-task-analyzer
Autor	Pedro Vargas dos Santos e Silva
Curso	Ciência de Dados e Machine Learning
Disciplina	Bootcamp III (EAD — Disciplina Virtual)
Etapa	Fase 3 — Entrega Final (DevOps, Containerização e Governança Ética de IA)
Versão do contrato	1.1.0 (`specs/task_analyzer_spec.md`)
Data de entrega	25/10/2026

Objetivo e escopo

Este relatório audita o uso de inteligência artificial na construção do módulo TaskAnalyzer. Ele responde a três perguntas: **o que impediu a IA de produzir código fora do contrato**, **onde o julgamento humano foi exercido** e **o que precisa ser preservado para que o resultado continue auditável**.

O escopo é o próprio repositório: as regras de `CONTEXT_RULES.md`, a especificação em `specs/task_analyzer_spec.md`, a suíte em `tests/test_harness.py`, a implementação em `src/task_analyzer.py` e a infraestrutura adicionada nesta fase (`.github/workflows/ci.yml`, `Dockerfile`, `docker-compose.yml`).

Convenção de honestidade adotada neste documento. Cada afirmação é classificada como:

- **Fato verificável** — observável no repositório ou na execução dos testes.
- **Proposta** — procedimento recomendado, ainda não executado.
- **Pendência** — lacuna reconhecida, com o risco associado.

Nenhuma aprovação, execução ou métrica é declarada sem evidência correspondente.

Seção A — Análise de alucinações

A.1 O que significa "alucinação" neste projeto

No contexto de geração de código, alucinação não é apenas inventar uma biblioteca inexistente. A manifestação mais perigosa é mais sutil: **produzir código plausível, sintaticamente válido e executável, que implementa uma regra de negócio que ninguém pediu**. Esse tipo de erro passa por revisão superficial, porque o código "parece certo".

O projeto trata esse risco com três camadas distintas, de naturezas diferentes.

A.2 Instruções contextuais versus regras executáveis

A distinção é central para entender o que cada camada pode e não pode garantir.

Camada	Artefato	Natureza	Força
1	`CONTEXT_RULES.md`	Instrução em linguagem natural	Probabilística: orienta o modelo, mas não impede nada mecanicamente
2	`specs/task_analyzer_spec.md`	Contrato de negócio com assinaturas literais	Normativa: define o certo e o errado de forma objetiva
3	`tests/test_harness.py`	Código executável	Determinística: falha, e a falha é visível

A camada 1 **influencia** a geração. As camadas 2 e 3 **verificam** o resultado. Confiar apenas na camada 1 seria confiar que o modelo obedece — o que não é garantido. Por isso a especificação estabelece, na sua seção 3, uma ordem de precedência explícita: especificação → testes → regras de contexto → instruções pontuais da conversa.

A Fase 3 adiciona uma quarta camada, de natureza organizacional: o CI do GitHub Actions executa a camada 3 **fora da máquina do desenvolvedor**, a cada push e a cada Pull Request nas branches `main` e `develop`. Isso remove a possibilidade de um resultado verde depender de um ambiente local particular.

A.3 Como os contratos limitam interpretações indevidas

Fatos verificáveis, extraídos do repositório:

Nomes literais (regras D08, P05 e P11). A seção 1.3.5 da especificação fixa as assinaturas públicas: `analyze_tasks`, `TaskValidationError`, `tempo_medio_conclusao_horas`, `total_tarefas`. Um campo inventado ou renomeado quebra os testes imediatamente, porque as asserções referenciam esses nomes diretamente. Não há espaço para "equivalente razoável".

Domínios fechados (RN12). `Prioridade` e `Status` são `StrEnum` com três valores cada. Um status "em_andamento" plausível, mas não especificado, é rejeitado por `_validar_dominio` e verificado pelo cenário CE-06.

Dependências restritas (P01 e RNF02). `requirements.txt` contém apenas `pytest` e `pytest-cov`, com versões fixadas. O módulo `src/task_analyzer.py` importa exclusivamente da biblioteca padrão: `logging`, `dataclasses`, `datetime`, `enum` e `typing`. A tendência de recorrer a `pandas` ou `python-dateutil` para manipular datas foi bloqueada na origem.

Proibição de alterar testes (P02). Quando um teste falha, a correção deve ocorrer no código. Essa regra existe porque ajustar a asserção é o caminho de menor resistência e destrói o valor do Test Harness. A verificação é feita no diff do Pull Request.

Tratamento de divisão por zero (RN10 e D13). Este é o exemplo mais instrutivo. A regra não se limita a dizer "evite divisão por zero": ela determina **como**. A guarda é um `if quantidade == 0: return 0.0`, nunca um `try/except ZeroDivisionError`. A diferença é semântica: a guarda declara que ausência de dados é um estado previsto do domínio, e não uma exceção a ser capturada. Sem essa precisão, o modelo escolheria sozinho, e qualquer das duas escolhas pareceria defensável na revisão.

Minimização de dados em log (D07 e RNF05). O campo `titulo` nunca é registrado em log. A regra é verificada pelo cenário CA-11, que injeta um título sabidamente sensível e confirma, via `caplog`, que ele não

aparece em nenhuma saída de log.

A.4 Como os testes detectam violações

A suíte traduz os 27 cenários de aceite da especificação em **37 casos de teste** executáveis, com nomes rastreáveis (`test_ca01_*`, `test_ce01_*`), de modo que a falha aponta diretamente para a regra violada. O arquivo `specs/requirements.yaml`, criado nesta fase, formaliza esse vínculo: cada regra RN01 a RN16 lista os testes que a verificam e sua situação de validação.

Fato verificável: a execução local de `pytest tests/test_harness.py -v` na preparação desta entrega resultou em **37 passed**, e `--cov=src` reportou **100% de cobertura** de `src/task_analyzer.py`. A cobertura total é uma evidência adicional relevante: indica que **não há caminhos de código sem cenário correspondente** — ou seja, a IA não acrescentou lógica além do contrato.

A.5 Caso real: o conflito entre o enunciado e a especificação

Fato verificável, registrado no histórico de versões da especificação e no Pull Request #1 do repositório.

O enunciado da Fase 2 exigia comportamentos que divergiam da especificação aprovada na Fase 1 em oito pontos, entre eles: o nome da função (`analyze_tasks` contra `analisar_tarefas`), a unidade de tempo (horas contra minutos) e, sobretudo, o tratamento de lista vazia (retorno `0.0` contra lançamento de exceção).

Havia, portanto, **duas fontes da verdade conflitantes** — exatamente a condição em que um agente de IA preenche a lacuna por conta própria, e o faz de forma invisível. A regra I05 ("se uma instrução conflitar com este arquivo ou com a especificação, aponte o conflito antes de agir"), combinada com P08 ("não assuma comportamentos não especificados; havendo ambiguidade, PARE e pergunte"), produziu o comportamento desejado: o conflito foi explicitado em uma tabela de decisões, submetido a decisão humana e resolvido com a publicação da versão 1.1.0 da especificação **antes** de qualquer linha de código ser gerada.

A lição é metodológica: o ganho não veio de uma regra que descreve o código correto, mas de uma regra que obriga o agente a **declarar incerteza** em vez de resolvê-la sozinho.

A.6 Riscos de alucinação que permanecem

Este projeto **não elimina** alucinações. Ele reduz a superfície de risco e torna detectáveis as violações que ocorrem dentro do escopo contratado. Permanecem em aberto:

Risco	Por que persiste	Mitigação atual	Status
Erro dentro do contrato	O teste verifica o que foi especificado; uma regra de negócio mal concebida é implementada fielmente	Revisão humana da própria especificação	Parcial
Cenário não especificado	O que não está na spec não é testado (ex.: comportamento com 10 milhões de tarefas)	Cobertura de 100% indica ausência de código extra, não ausência de lacuna na spec	Parcial
Código plausível porém supérfluo	Abstração desnecessária que passa nos testes	Regras D04 (responsabilidade única) e P07 (sem código morto), verificadas por leitura humana	Dependente de revisão
Documentação divergente do código	Docstrings e comentários não são executados	Revisão humana; nenhuma verificação automatizada	**Pendência**
Dependência de uma única revisão	Projeto individual: autor e revisor são a mesma pessoa	Checklist escrito reduz, mas não remove, o viés	**Pendência estrutural**

Pendência declarada: não existe análise estática automatizada (`ruff`, `mypy`, `bandit`) no pipeline. Ferramentas desse tipo detectariam classes inteiras de problemas sem depender da atenção do revisor. A ausência é uma limitação consciente, decorrente da restrição de dependências do contrato, e está entre as recomendações finais.

Seção B — Homologação humana: o modelo Critical-in-the-Loop

B.1 Por que "critical" e não apenas "human-in-the-loop"

A expressão *human-in-the-loop* descreve a presença humana no processo. Ela é compatível com uma presença passiva: aprovar o que a máquina produziu porque os testes ficaram verdes. O modelo adotado aqui é mais exigente — o humano atua como **filtro primário e homologador crítico**, com o ônus de encontrar o que os testes não encontram.

A distinção é operacional: testes automatizados respondem **"o código faz o que o contrato diz?"**; o homologador humano responde **"o contrato diz o que o negócio precisa?"** e **"esse código é sustentável?"**. A segunda pergunta não é automatizável.

B.2 O fluxo em quatro etapas

Etapa	Responsável	O que acontece	Evidência no repositório
1. Geração	Agente de IA	Implementa a lógica a partir da spec e das CONTEXT_RULES	Commit `feat:` com rodapé `AI-Assisted`
2. Barreira SDD	Automação	Código passa pelo contrato executável e pela suíte de testes	Execução do CI na aba Actions
3. Validação humana	Desenvolvedor	Revisa o diff, procura inconsistências que os testes não capturam	Checklist no Pull Request
4. Integração segura	Desenvolvedor	Merge na `main` após as etapas 1 a 3	PR com selo *Merged*

A etapa 3 é a única em que uma objeção pode ser levantada sem que nada tenha falhado tecnicamente. É ali que mora o valor do modelo.

B.3 Critérios de aceitação

Derivados da seção 4.1 da especificação. O código só é integrado quando **todos** são satisfeitos:

1. **Testes.** `pytest tests/test_harness.py -v` com 100% dos casos aprovados e cobertura de `src/` maior ou igual a 90%.
2. **Conformidade contratual.** Assinaturas idênticas à seção 1.3.5 da especificação; nenhum campo, chave ou tipo fora do contrato (P11).
3. **Qualidade.** Ausência de `print`, de `except Exception` genérico e de imports externos; type hints em todas as funções; docstrings no padrão Google; funções com responsabilidade única.
4. **Verificação manual.** Execução de casos adicionais fora da suíte, anotados no PR.
5. **Aprovação explícita.** Merge do Pull Request, com o commit identificado como assistido por IA.

Critério de rejeição. Teste falhando, violação de qualquer proibição P01 a P12, ou comportamento não especificado implicam devolução do código ao agente com o motivo explícito — e não correção manual silenciosa, que destruiria a rastreabilidade entre o prompt e o resultado.

B.4 Tratamento de falhas, ambiguidades e resultados inesperados

Situação	Procedimento	Natureza
Teste falha	Corrigir o código, nunca o teste (P02). Se o teste estiver errado, corrigir primeiro a especificação e depois o teste	Fato: aplicado na Fase 2
Requisito ambíguo	O agente para e pergunta (P08). A resposta é incorporada à especificação, com nova versão	Fato: originou a v1.1.0
Conflito entre fontes	O agente aponta o conflito (I05); a decisão é humana e fica registrada no histórico de versões	Fato: oito decisões registradas
Resultado inesperado porém aprovado nos testes	Investigar se a spec está incompleta; a correção começa pela especificação	Proposta
CI vermelho	Nenhum merge até o build voltar ao verde	Proposta operacional da Fase 3

B.5 Checklist de homologação da Fase 3

Proposta de checklist a ser preenchido no Pull Request desta fase. Os itens marcados como executados referem-se a verificações já realizadas na preparação da entrega; os demais dependem de execução no ambiente do autor.

Verificações já realizadas (fatos):

- ✓ `pytest tests/test_harness.py -v` — 37 casos aprovados.
- ✓ Cobertura de `src/` em 100%, acima do mínimo de 90%.
- ✓ Sintaxe de `ci.yml` e `docker-compose.yml` validada como YAML.
- ✓ `specs/requirements.yaml` validado: as 16 regras estão presentes, em ordem, e todos os testes e funções citados existem de fato no repositório.
- ✓ Nenhuma credencial, token ou dado pessoal sensível incluído nos arquivos.

Verificações que dependem do ambiente do autor (pendências):

- ■ Build verde confirmado na aba Actions do GitHub, após o push.
- ■ `docker build -t task-analyzer-tests .` executado com sucesso.
- ■ `docker compose run --rm tests` executado com sucesso.
- ■ Revisão visual dos PDFs gerados.
- ■ Pull Request `develop` → `main` aberto, revisado e mergeado.

B.6 Limitação estrutural reconhecida

Este é um trabalho individual: **o autor do contrato, o operador do agente e o homologador são a mesma pessoa**. Em ambiente profissional, a separação entre quem especifica e quem aprova é o que confere independência à homologação. Aqui, essa independência não existe.

A mitigação possível é documental: critérios escritos **antes** da geração do código, de modo que a aprovação seja conferida contra um padrão fixado previamente, e não construída a posteriori para justificar o que foi produzido. É uma mitigação parcial, e registrá-la como tal é parte da honestidade metodológica exigida pela governança.

Seção C — Sustentabilidade, ética e rastreabilidade dos prompts

C.1 Por que versionar prompts

No desenvolvimento assistido por IA, o prompt é **insumo de produção**. Um repositório que versiona o código gerado mas descarta as instruções que o produziram guarda o resultado e perde a causa: torna-se impossível auditar por que o sistema se comporta de determinada maneira, ou reproduzir a geração sob as mesmas condições.

No TaskAnalyzer, o contexto fornecido ao agente é versionado em dois arquivos: a especificação, que define o **o quê**, e o `CONTEXT_RULES.md`, que define o **como**. Ambos estão sob controle de versão, com histórico de alterações e commits próprios.

C.2 Cadeia de rastreabilidade

Fato verificável. É possível percorrer o caminho completo em ambos os sentidos:

```
CONTEXT_RULES.md (regra D13: guarda explícita contra divisão por zero)
        v
specs/task_analyzer_spec.md (RN10: sem concluídas, médias valem 0.0)
        v
specs/requirements.yaml (RN10 -> cenários CA-09 e CA-10)
        v
tests/test_harness.py (test_ca09_lista_vazia_retorna_zeros)
        v
src/task_analyzer.py (funções _media e _percentual)
        v
.github/workflows/ci.yml (execução automática a cada push e PR)
```

A direção inversa também funciona: diante de uma linha do código, é possível identificar a regra que a originou. Essa propriedade é o que torna o sistema **auditável**, e não apenas testado.

C.3 Estrutura proposta para registro de alterações em prompts

Proposta. O projeto versiona os prompts por meio do `CONTEXT_RULES.md` e do histórico da especificação, mas não mantém um registro dedicado de alterações no contexto fornecido à IA. Recomenda-se, para a continuidade do trabalho, um arquivo `docs/prompt_changelog.md` com a estrutura abaixo:

Campo	Conteúdo
Versão	Identificador incremental (ex.: `prompt-v1.2`)
Data	Data da alteração
Objetivo	O que se pretendia obter ou corrigir
Mudanças	Regras acrescentadas, removidas ou reescritas
Justificativa	Problema observado que motivou a mudança
Testes afetados	Cenários que passaram a existir, mudar ou falhar
Resultado	O que efetivamente mudou no código gerado

Exemplo preenchido com um caso real deste projeto:

Campo	Conteúdo
Versão	prompt-v1.1
Data	Fase 2
Objetivo	Resolver o conflito entre o enunciado da Fase 2 e a especificação v1.0.0
Mudanças	Oito decisões registradas: renomeação da função, nova base de exceções, mudança de unidade e de política para lista vazia
Justificativa	Duas fontes da verdade conflitantes, condição propícia a decisão silenciosa do agente
Testes afetados	CE-02 e CE-03 deixaram de ser cenários de erro e passaram a ser CA-09 e CA-10, de sucesso
Resultado	Contrato único antes da geração; código gerado sem ambiguidade

C.4 Riscos éticos identificados

Viés. O TaskAnalyzer calcula métricas de produtividade — tempo médio de conclusão e taxa de atraso — que podem ser usadas para avaliar pessoas. O módulo é deliberadamente neutro quanto ao uso: não classifica, não ranqueia e não emite juízo. Porém, quem consome essas métricas pode atribuir-lhes significado que os dados não sustentam. Uma tarefa concluída com atraso pode refletir subdimensionamento do prazo, dependência externa ou prioridade alterada — nenhuma dessas causas é capturada pelo modelo de dados.

Decisão de projeto registrada: quando não há tarefas concluídas, as médias retornam `0.0`, e a especificação determina explicitamente, na nota da RN10, que esse valor significa **"sem dados"** e não **"desempenho zero"**. A distinção é feita pelo campo `total_concluidas == 0`. Confundir as duas leituras produziria avaliação injusta de uma equipe que simplesmente não concluiu nada no período medido.

Opacidade. O módulo é determinístico e auditável: dada a mesma entrada, produz sempre a mesma saída, e cada valor pode ser rastreado até a regra que o originou. Não há modelo estatístico, inferência probabilística ou caixa-preta na execução. A IA participou da **construção** do sistema, não da sua **operação** — distinção relevante, porque elimina a opacidade em tempo de execução.

Dependência excessiva de IA. É o risco mais concreto em um trabalho acadêmico. A mitigação adotada tem três elementos: a especificação e os cenários de aceite foram escritos por humano antes da geração; os testes são intocáveis pelo agente (P02); e a aprovação final é humana e registrada. Ainda assim, cabe reconhecer: quem escreve a especificação precisa dominar o domínio o suficiente para avaliar o código gerado. **A IA reduz o esforço de digitação, não a necessidade de competência técnica.**

Proteção de dados. O módulo não persiste dados, não faz I/O e não acessa rede (RNF04). A regra D07 proíbe registrar o campo `titulo` em log, e o cenário CA-11 verifica essa proibição automaticamente. É minimização de dados aplicada no nível do código, em linha com o princípio da necessidade previsto no artigo 6º da LGPD.

Fato verificável: nenhum token, senha, chave de API ou dado pessoal de terceiros está presente no repositório. O workflow de CI não requer segredos, pois executa apenas testes sobre dados sintéticos construídos nas próprias fixtures.

C.5 Sustentabilidade computacional

Decisões tomadas nesta fase com impacto direto no consumo de recursos:

Decisão	Efeito
Imagem-base `python:3.11-slim`	Imagem substancialmente menor que a `python:3.11` completa: menos download, menos armazenamento, build mais rápido
`.dockerignore` restritivo	Documentação, PDFs, histórico Git e caches ficam fora do contexto de build
Dependências instaladas antes do código no `Dockerfile`	A camada de dependências é reaproveitada do cache quando apenas o código muda
`pip install --no-cache-dir`	Evita armazenar o cache do pip dentro da imagem final
`concurrency` com `cancel-in-progress` no CI	Execuções antigas da mesma branch são canceladas quando um novo push chega: não se gasta runner com código já superado
`cache: pip` no `setup-python`	Dependências não são rebaixadas da rede a cada execução
Suíte executando em cerca de 1 segundo	Permite rodar os testes localmente antes do push, reduzindo execuções desnecessárias no CI

O princípio orientador é simples: **cada execução de pipeline consome energia em um datacenter**. Em um projeto acadêmico o volume é irrelevante; o hábito, não. A mesma decisão replicada em escala industrial tem impacto material.

C.6 Limitações do projeto

Pendências declaradas, sem mitigação implementada:

1. **Sem análise estática no pipeline.** `ruff`, `mypy` e `bandit` detectariam problemas que a revisão humana pode deixar passar. Não foram incluídos por força da restrição de dependências (P01) e da ausência de exigência na atividade.
2. **Sem teste de desempenho automatizado.** O requisito RNF03 (complexidade O(n)) é verificado por leitura de código e por um teste manual com 1.000 tarefas registrado no PR da Fase 2, não por benchmark em CI.
3. **Revisão por um único indivíduo.** Discutido na seção B.6.
4. **Sem registro dedicado de alterações de prompts.** A estrutura está proposta na seção C.3, mas não implementada neste ciclo.
5. **Docker não validado neste ambiente.** Os arquivos foram revisados estaticamente e o YAML validado, mas a execução de `docker build` depende de uma máquina com Docker instalado. Os comandos exatos constam do README.

Conclusão

A Fase 3 completa um arco que começou com um documento de texto e termina com um sistema que se verifica sozinho a cada alteração. O percurso sugere três conclusões.

Primeira: a especificação é o verdadeiro artefato de engenharia. O código foi gerado em minutos; a especificação levou semanas e precisou ser revisada quando o requisito mudou. A qualidade do resultado é consequência direta da qualidade do contrato — e, quando havia duas fontes de verdade conflitantes, nenhuma regra de estilo teria evitado o problema. O que evitou foi a regra que obrigou o agente a **declarar o conflito** em vez de resolvê-lo por conta própria.

Segunda: automação e governança são camadas complementares, não substitutas. O CI garante que o código faz o que o contrato diz. Não garante que o contrato esteja certo. Essa segunda pergunta permanece humana, e qualquer narrativa que sugira o contrário é falsa.

Terceira: a honestidade sobre as limitações é parte da entrega. Este relatório registra cinco pendências explícitas e duas limitações estruturais. Um relatório de governança que declarasse o problema resolvido seria, ele próprio, uma forma de alucinação — produzir um texto plausível sobre um estado de coisas que não corresponde à realidade verificável.

Recomendações para evolução

1. Acrescentar `ruff` e `mypy` como etapa não bloqueante do CI, evoluindo para bloqueante.
2. Implementar o `docs/prompt_changelog.md` proposto na seção C.3.
3. Publicar o relatório de cobertura como artefato do CI, permitindo acompanhar a evolução.
4. Em contexto profissional, exigir revisor distinto do autor para código gerado por IA.
5. Incluir um teste de carga com volume representativo para verificar a RNF03.

Referências

As referências abaixo são normas e publicações públicas, citadas apenas onde embasam afirmações deste relatório. Nenhuma fonte foi criada para este documento.

1. BRASIL. **Lei nº 13.709, de 14 de agosto de 2018** — Lei Geral de Proteção de Dados Pessoais (LGPD). Princípio da necessidade, art. 6º, III. Fundamenta a seção C.4.
2. NIST. **Artificial Intelligence Risk Management Framework (AI RMF 1.0)**. National Institute of Standards and Technology, 2023. Funções *Govern*, *Map*, *Measure* e *Manage*; embasa a estrutura das seções A e B.
3. ISO/IEC. **ISO/IEC 42001:2023 — Information technology, Artificial intelligence, Management system**. Requisitos de sistema de gestão de IA, incluindo rastreabilidade e documentação de decisões.
4. UNIÃO EUROPEIA. **Regulamento (UE) 2024/1689** — Regulamento da Inteligência Artificial. Requisitos de supervisão humana e de registro, referidos na seção B.
5. VAN ROSSUM, G.; WARSAW, B.; COGHLAN, N. **PEP 8 — Style Guide for Python Code**. Python Software Foundation. Base das diretrizes D03 e D05 do `CONTEXT_RULES.md`.
6. SILVA, Pedro Vargas dos Santos e. **Especificação SDD — TaskAnalyzer, versão 1.1.0**. `specs/task_analyzer_spec.md`, repositório do projeto.

Documento elaborado com assistência de IA para redação e estruturação, a partir de evidências extraídas do repositório, e revisado integralmente pelo autor. As classificações entre fato, proposta e pendência refletem o estado do projeto na data de entrega.